

MapPublisher 再構成・ハイキングマップデータ公開移行 実装計画

版: 1.7 / 作成日: 2026-08-18 / 最終更新: 2026-08-23

対象リポジトリ: **MapPublisher** / **MapEditor** / **minoh-hiking** 対象契約バージョン: **2.0**

本書の守備範囲は契約 2.0 までの移行である。ClosureEditor の再構成と、ハイキングマップデータを公開経由の配信へ移す作業を記録している。これより後の変更は本書に反映しない。現在の仕様は次の3文書が正本である。

変更	契約	正本
<code>tiles</code> （地図タイルのダウンロード領域）の追加	2.1	機能仕様 §3.5 / データ仕様 §5
画面を開閉式に変更、出力を公開済みデータへ変更、失敗時の復元	2.1	機能仕様 §2.1・§3.7.3・§3.8

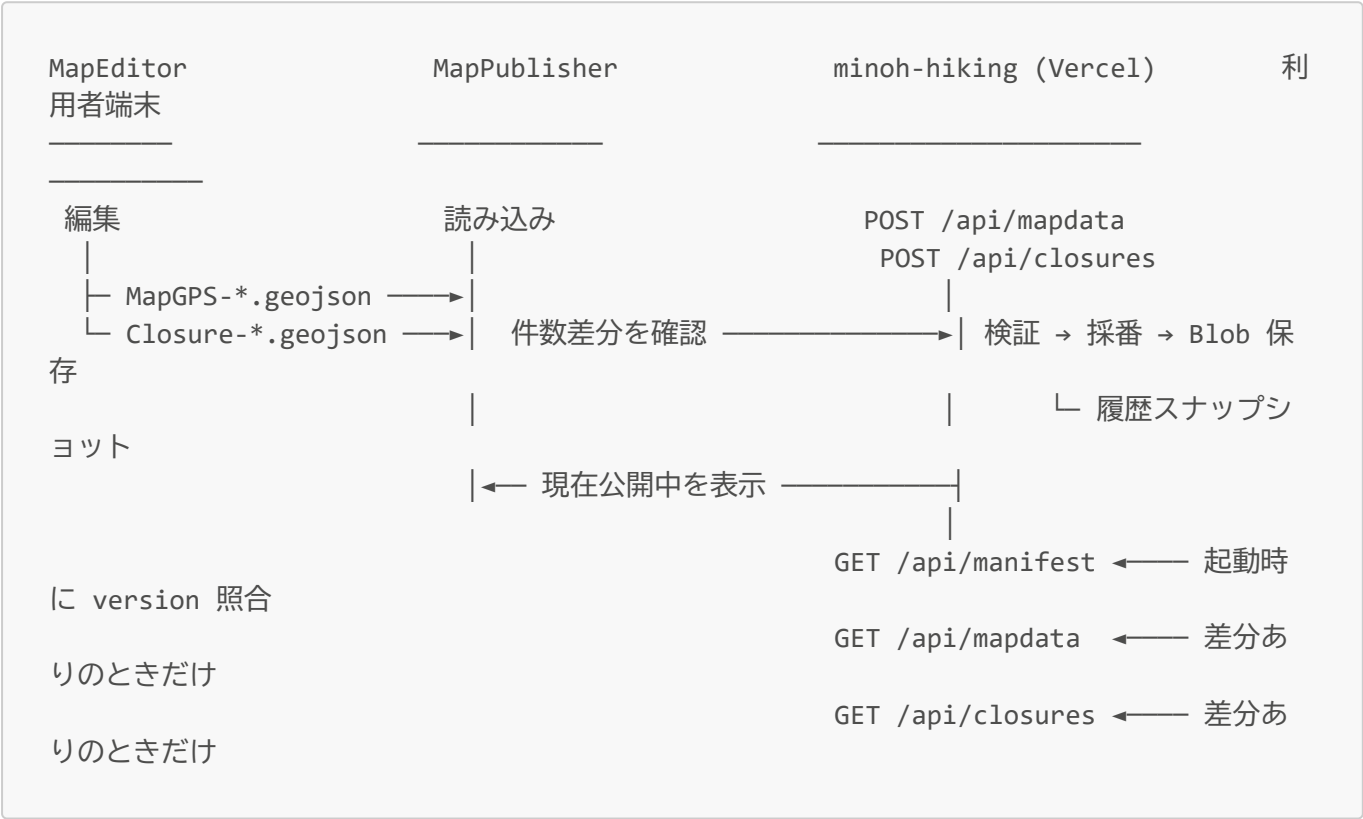
0. 決定事項

#	項目	決定
1	公開対象	通行止め・通行困難地点に加え、 ポイント・ルート・スポットも公開する
2	データセット数	2つ 。 <code>mapdata</code> （緊急ポイント＋ルート＋スポット） / <code>closures</code> （通行止め・通行困難） ※ 契約 2.1 で <code>tiles</code> を追加し3つになった（本計画の対象外）
3	version 採番	サーバー側で自動採番 。 <code>mapdata</code> は <code>yyyy.n</code> 、 <code>closures</code> は <code>yyyy-mm.n</code>
4	n の桁	1桁 （ゼロ埋めしない）
5	公開トークン	<code>MAP_PUBLISH_TOKEN</code> に統一 （2データセット共通）
6	更新判定	起動時に version を読み、 更新版があり、かつ通信可能なときだけデータ を取得する
7	オフライン表示	前回取得した <code>geojson</code> を保持して表示する 。真の初回オフライン起動は表示なしで可（地理院タイルも無いため）
8	更新順	本体の取得・保存が成功してから <code>version</code> を保存する
9	履歴スナップショット	前回分のみ保持する （1世代・上書き）
10	誤公開の件数下限チェック	実装しない 。機能仕様書 §5「実装しないこと」の方針を維持する

役割分担（確定）

リポジトリ	役割
MapEditor	地点・ルート・スポットの編集、通行止め地点の登録。geojson ファイルを出力する
MapPublisher	geojson ファイルを読み込み、 公開する 。編集機能は持たない
minoh-hiking	公開API の配信元、および利用者向け表示アプリ

1. 全体像



2. 公開データの定義

2.1 mapdata（統合）

現行の minoh-hiking-routes-spots.geojson と minoh-emergency-points.geojson を**1本に統合**する。

type	件数	扱い
ポイントGPS	167	公開する（緊急ポイントとして表示）
route	292	公開する
spot	209	公開する
point	167	公開しない（除去）
area	0～	公開しない（除去）
公開対象計	668	

point を除去して安全な根拠（検証済み）：

- route が端点として参照する ID 214件の提供元は **ポイントGPS** 166件 / 未解決48件で、**point** 由来はゼロ
- **point** は minoh-hiking の `buildHikingLayer` が `type === 'spot'` で絞るため一度も描画されていない

emergency-points を統合して安全な根拠（検証済み）：

- **emergency-points** 167件は **routes-spots** 内の **ポイントGPS** 167件と **ID集合・座標とも完全一致**（差分ゼロ）

2.2 公開スキーマ（利用者の表示に必要なものだけを出す）

正本は minoh-hiking docs/publish-api-202608.md §3。 以下は計画立案時の要約であり、相違がある場合は仕様書が優先する。

公開データは表示専用であり、編集用の識別子は不要である（§9.2）。MapPublisher が公開時に以下へ整形する。

type	出力するプロパティ	出力しないもの	理由
ポイントGPS	type / id / name / description（非nullのみ）	pointId	全167件で id と同値
spot	type / name	id / source / description	id はどこからも参照されていない。source は全209件が image_transformed、description は全209件が スポット（GPS変換済）の定数
route	type / id / startPointGPS / endPointGPS	startPoint / endPoint	端点座標が入っており ID 参照は不要。区間の識別は id（route_H-04_to_H-11）で足りる

- **ポイントGPS** の id（B-01 等）は**現地の標識と対応する利用者向けの識別子**のため残す
- **description** が非nullなのは **ポイントGPS** の12件のみ

2.3 配信サイズ

	minified	削減
現状（routes-spots 835件 + emergency 167件の2ファイル）	319,764 B	—
統合・point 除去（668件・全プロパティ）	248,519 B	-22.3%
+ spot の id / source / description 除去	223,199 B	-30.2%
+ pointId / null description 除去	220,360 B	-31.1%
+ route の startPoint / endPoint 除去（公開スキーマ）	208,478 B	-34.8%

加えて `public/data/*.geojson`（実ファイル 704KB）がアプリシェルから外れるため、**初回起動時のダウンロード量が減る。**

2.4 検証で判明した実データの状態

項目	実測値	対応
座標範囲	経度 135.45278~135.50664 / 緯度 34.83143~34.88649	現行API の <code>LON_RANGE[135.2,135.8]</code> / <code>LAT_RANGE[34.6,35.1]</code> に収まる。 流用可
ID 欠落	0件	問題なし
ID 重複	<code>spot07_WC</code> が2件	公開スキーマで <code>spot</code> の <code>id</code> を出力しないため解消 (§9.3)
spot 名の重複	12種・17件 (トイレ×5、WC×4 ほか)	名称の重複は許容する。識別は座標で行う

2.5 closures

現行どおり。Point のみ、`kind` は `closed` / `difficult`。

3. version 採番仕様

3.1 形式

データセット	形式	例
mapdata	<code>yyyy.n</code>	<code>2026.1</code>
closures	<code>yyyy-mm.n</code>	<code>2026-08.1</code>

- `n` は1桁を前提とし、**ゼロ埋めしない**
- 期間 (mapdata は年、closures は年月) が変われば `n` は 1 に戻る

3.2 採番規則

1. 現在公開中の version を manifest から読み、`yyyy / yyyy-mm` と `n` に分解する
2. **サーバー時計を JST (Asia/Tokyo) に変換**して現在の期間を求める
 - Vercel Functions は UTC で動作する。UTC のまま判定すると、**9月1日 08:00 JST の公開が 8月扱いになるため必ず変換すること**
3. 期間が変わっていれば `n = 1`、同じなら `n = 現在値 + 1`
4. 公開中の期間がサーバー時計より**未来**の場合 (時計ずれ) は、期間を戻さず `n` だけ加算する
5. version がパースできない場合 (旧・手入力形式) は、現在の期間で `n = 1` から開始する

3.3 比較は等値のみ

`n` をゼロ埋めしないため、文字列の大小比較は `2026.10 < 2026.9` と逆転する。

- **更新判定は `!==` (等値比較) のみで行い、大小比較を実装しない**
- `n` が 10 に達した場合は2桁で継続する。動作に影響はなく、履歴ファイル名の並び順のみが崩れる

4. 公開API 仕様 (契約バージョン 2.0)

正本は minoh-hiking docs/publish-api-202608.md。 以下は計画立案時の要約であり、相違がある場合は仕様書が優先する。検証ルール・エラー文言は本書に書き写さないこと。

4.1 エンドポイント

メソッド	パス	認証	用途
GET	<code>/api/manifest</code>	不要	全データセットの version・件数を返す（数百バイト）
GET	<code>/api/mapdata</code>	不要	統合geojson を返す
POST	<code>/api/mapdata</code>	要	公開（全置換）
GET	<code>/api/closures</code>	不要	既存どおり
POST	<code>/api/closures</code>	要	公開（全置換）。 version はサーバー採番に変更

GET `/api/manifest` の応答:

```
{
  "mapdata": { "version": "2026.1", "updatedAt": "...", "count": 668 },
  "closures": { "version": "2026-08.1", "updatedAt": "...", "count": 12 }
}
```

4.2 Blob 構成

manifest.json	← 採番の基準・GET <code>/api/manifest</code> の実体
mapdata/minoh-hiking-mapdata.geojson	← 現行
mapdata/previous.geojson	← 前回分（1世代のみ・毎回上書き）
closures/minoh-hiking-closure.geojson	← 現行（既存パスを維持）
closures/previous.geojson	← 前回分（1世代のみ・毎回上書き）

履歴は**前回分1世代のみ**とする（決定事項 #9）。現行の `closures/history/` は世代を無制限に増やすため、Phase 5 で削除する。

4.3 POST の書き込み順（重要）

1. `manifest.json` を読み、採番する
2. 検証する
3. 現行の本体を `previous` へ退避する
4. 本体 blob を put する
5. `manifest.json` を put する
6. 200 を返す

手順5で失敗した場合は **500** を返す。manifest が進んでいないため、再実行すると同じ **version が再採番される（冪等）**。運用者は「もう一度公開」で復旧できる。

手順3の実装:

```
// @vercel/blob@2.6.1 の copy() を使う。本体をダウンロードせずに退避できる
await copy(BLOB_PATH, PREVIOUS_PATH, {
  access: 'public',
  contentType: 'application/geo+json', // copy は metadata を引き継がないため再指定が必要
  addRandomSuffix: false,
  allowOverwrite: true
});
```

- 初回公開時は本体が存在せず `BlobNotFoundError` になる。**握りつぶして続行する**
- 退避に失敗しても**公開は成立させる**（警告ログのみ）。公開を止めるほうが運用上の害が大きい
- 手順3が成功して手順4が失敗した場合、`previous` は現行と同一内容になるだけで害はない

4.4 検証ルール

共通（既存 `validateClosureGeoJSON` から流用）：

- `FeatureCollection` 形式であること
- 座標が箕面エリアの範囲内であること（`LineString` は**全頂点**を検査する）
- `id` が一意であること — 既存実装は `if (id !== null)` で未設定をスキップするため、`id` を持たない `spot` (§2.2) は**検証コードを変更せずにそのまま通る**

mapdata 固有：

- `properties.type` が `ポイントGPS` / `route` / `spot` のいずれかであること（それ以外は 400）
- `Point` と `LineString` の両方を受け付けること

現行の `validateClosureGeoJSON` は `if (f.geometry.type !== 'Point') return エラー` で **`LineString` を必ず弾く** (`api/closures.js:161`)。さらに `const [lon, lat] = f.geometry.coordinates` (同:164) は `LineString` だと `lon` が配列になり 400 になる。**mapdata 用に別の検証関数を用意すること**。closures 側の関数是不変。

closures 固有: 現行どおり（`Point` のみ）。

4.5 トークン

- 環境変数を `MAP_PUBLISH_TOKEN` に統一する
- ヘッダ名 `x-publish-token` は変更しない
- 検証・CORS・採番・Blob 操作は `api/_lib/` に切り出し、2エンドポイントで共有する

5. minoh-hiking アプリ側

5.1 起動フロー

1. `localStorage` から保存済み `version` を読む
2. `Cache API` から `geojson` を読んで即描画 ← オフラインでもここまでで表示される
3. `GET /api/manifest (cache: 'no-store')`
 - └ 失敗（オフライン等） → 終了

```

└─ version が一致          → 終了（本体を取りに行かない）
└─ version が相違
    4. GET 本体
    5. Cache API に保存
    6. 再描画
    7. localStorage の version を更新 ← ★必ず最後

```

手順7を先に行ってはならない。本体の取得や保存に失敗したあとに version だけが進むと、以後その端末は永久に更新されなくなる。

5.2 キャッシュの責務をアプリ側へ一本化

現行の Service Worker は `/api/closures` を `network-first` で処理している (`handleClosureRequest`)。version ゲート方式とは噛み合わないため、**SW は `/api/*` を素通しにし、キャッシュ制御はアプリ側に集約する**。SW はアプリシェルと地理院タイルのみを担当する。

5.3 修正一覧

ファイル	修正内容
<code>public/config.js</code>	<code>EMERGENCY_URL</code> / <code>HIKING_ROUTES_URL</code> を廃止し、API URL と manifest URL を追加
<code>public/app.js</code>	起動フローを §5.1 に変更
<code>public/map.js</code>	<code>buildEmergencyLayer</code> に <code>type === 'ポイントGPS'</code> の filter を追加 (§9.1)
<code>public/map.js</code>	<code>withRouteEndpoints</code> を <code>startPointGPS</code> / <code>endPointGPS</code> 参照に変更 (§9.2)
<code>public/map.js</code>	<code>buildEndpointIndex</code> を削除（不要になる） (§9.2)
<code>public/map.js</code>	<code>buildHikingLayer</code> の spot ポップアップを name のみ に変更 (§9.2)
<code>public/map.js</code>	<code>getFeatureCounts</code> を統合データ前提の集計に変更
<code>public/map.js</code>	<code>loadEmergencyPointsLayer</code> 内の <code>rebuildHikingLayer()</code> （読み込み順の race 対策）が不要になるため削除
<code>public/service-worker.js</code>	<code>/api/*</code> の横取りを削除。 <code>data/*.geojson</code> を <code>SHELL_LOCAL_PATHS</code> から除外。 activate の掃除除外リストとキャッシュ名を更新 (§5.4)
<code>public/data/*.geojson</code>	削除（Phase 5。1リリース遅らせる）

5.4 実装時に決めておく値

計画時点で未決だった具体値をここで確定する。

項目	決定
Cache API のキャッシュ名	<code>mapdata-cache</code> / <code>closures-cache</code> （既存名を流用）

項目	決定
localStorage キー	<code>minoh-hiking.mapdata-version / minoh-hiking.closures-version</code> (既存の <code>minoh-hiking.*</code> 命名に合わせる)
<code>SHELL_CACHE</code> の名前	必ず更新する (<code>app-shell-yyyy-mm-dd.n</code>)。\$5.3 で <code>SHELL_LOCAL_PATHS</code> から <code>data/*.geojson</code> を外すため
<code>getFeatureCounts</code> の集計	ポイント = <code>type === 'ポイントGPS'</code> / ルート = <code>type === 'route'</code> / スポット = <code>type === 'spot'</code> を数える

★ Service Worker がアプリ管理のキャッシュを消す

`service-worker.js` の `activate` は、許可リストに無いキャッシュをすべて削除する。

```
keys.filter((k) => k !== SHELL_CACHE && k !== CLOSURE_CACHE &&
!k.startsWith(TILE_CACHE_PREFIX))
  .map((k) => caches.delete(k))
```

キャッシュ制御をアプリ側へ移す (§5.2) と、`mapdata-cache` はアプリが作り SW は関与しないキャッシュになる。このリストに追加し忘れると、SW が更新されるたびに削除される。

その場合、オフライン起動時に前回の geojson が読めず地図が出ない (決定事項 #7「前回取得した geojson を保持して表示する」が成立しなくなる)。しかもオンラインでは正常に見えるため、**気づきにくい**。

対応: 除外条件に `mapdata-cache` を加える。`CLOSURE_CACHE` は名前を流用するためそのまま残す。

6. MapPublisher 再構成

6.1 画面構成

1画面・2セクション。編集機能は持たない。

これは移行当時 (契約 2.0) の画面である。現在は3セクションの開閉式で、「現在公開中」は見出しにあり、出力ボタンは「公開済みデータを出力」になっている。現行の画面構成は [機能仕様 §2](#) を参照すること。

```
┌ ハイキングマップデータ ───────────┐
│ ファイル読み込み [消去]              │
│ ポイント 167 / ルート 292 / スポット 209 │
│ 現在公開中: 2026.1 (668件)             │ [?] │
│ [公開] [整形結果を出力]              │
└──────────────────────────────────┘

┌ 通行止め・通行困難地点 ───────────┐
│ ファイル読み込み [消去]              │
│ 通行止め 8 / 通行困難 4              │
│ 現在公開中: 2026-08.1 (12件)          │ [?] │
│ [公開] [整形結果を出力]              │
└──────────────────────────────────┘
```

[トークン消去]

読み込んだ両データセットを地図に重ねて表示する（確認用・読み取り専用）。

6.2 公開前の確認ダイアログ

version が自動採番になると「古いファイルを新 version で再公開」が version の見た目では判別できなくなる。**version 比較**に代えて、**type 別の件数差分を防波堤とする**。

ハイキングマップデータをユーザーへ公開します。

現在公開中： 2026.1
ポイント 167 / ルート 292 / スポット 209（計 668件）

これから公開： 2026.2（予定）
ポイント 167 / ルート 292 / スポット 210（計 669件） +1

よろしいですか？

いずれかの type が**減少**する場合は、その行を明示して警告する。

6.3 モジュール構成

ファイル	現状	想定	扱い
js/mapData.js	—	~300	新規（basemap.js を置換）。保持・type別集計・重複検出・公開用選別
js/publish.js	213	~300	2データセット対応・件数差分ダイアログ。version 送信は廃止
js/closureData.js	(508)	~180	closureEditor.js を表示専用に縮小・改名
js/render.js	—	~120	新規。マーカー描画を両データセットで共通化
js/fileIO.js	206	~140	applyLoadedVersion / getVersionInput を削除
js/constants.js	66	~90	API URL 3本・公開対象 type・トークンキー
js/app.js	105	~80	2セクションの配線のみ
js/mapCore.js	65	65	変更なし
js/message.js	26	26	変更なし
js/utils.js	91	~85	変更なし
js/basemap.js	178	—	削除（mapData.js へ）
js/closureEditor.js	508	—	削除（closureData.js へ）
js/elevation.js	27	—	削除（移動時の標高取得専用のため不要）

ファイル	現状	想定	扱い
<code>index.html</code>	140	~120	編集UI を削除、2セクション化
<code>styles.css</code>	364	~300	同上
計	1,989	~1,806	

6.4 削除する機能

- 追加・移動モード、マーカーのドラッグ、削除
- 名称・備考・区分・登録理由の編集
- 標高の自動取得
- version 入力欄（サーバー採番のため）
- 登録地点ドロップダウン（668件では機能しない。件数サマリに置換）

6.5 新たに必要な機能

- **読み込みは置換方式**とする。現行 `basemap.js` の `handleFilesSelected` は追加式で重複検出がなく、同じファイルを2回読むと `features` が2倍になる。公開は全置換であり、MapEditor は完全な1ファイルを出力するため、読み込みも「置換」に揃えるのが素直（id に依存した重複検出は不要になる）
- 公開スキーマへの整形 (§2.2)。 `point / area` の除去、 `spot` の `id / source / description` 除去、 `pointId` 除去、 `route` の `startPoint / endPoint` 除去
- type 別の件数集計。現行 `countFeatures` は geometry type でしか数えず、 `ポイントGPS / point / spot` を区別できない

7. 作業フェーズ

Phase	内容	対象	状態
0	契約2.0・公開スキーマの仕様化 → <code>publish-api-202608.md</code>	docs	完了
2	MapPublisher 再構成	MapPublisher	完了（Phase 1 に先行して実施）
1	公開API の実装。 <code>api/_lib/</code> 共通化、 <code>api/mapdata.js</code> ・ <code>api/manifest.js</code> 新規、 <code>api/closures.js</code> 改修	minoh-hiking	完了・デプロイ済み
3	ハイキングマップデータの初回公開	MapEditor → MapPublisher	完了
4	minoh-hiking アプリ側の切替	minoh-hiking	完了
5	後始末	全体	一部完了 (§7.2)

7.1 稼働状況（2026-08-20 時点）

GET `/api/manifest` の応答:

データセット	version	件数	公開日時
mapdata	2026.1	626 (ポイント 174 / スポット 188 / ルート 264)	2026-08-20T05:01:15Z
closures	2026-08.2	7 (通行止め 5 / 通行困難 2)	2026-08-20T05:15:06Z

配信データを検証し、公開スキーマ (§2.2) とおりであることを確認した。

```

ポイントGPS → type, id, name, description
spot          → type, name                    (id を出力していない)
route         → type, id, startPointGPS, endPointGPS (startPoint/endPoint を出力していない)

```

アプリ側の実装も確認済み。特に §5.4 で警告した **Service Worker によるアプリ管理キャッシュの削除**は、`APP_MANAGED_CACHES` (`mapdata-cache` / `closures-cache`) を掃除の除外に加えることで回避されている。version の保存も「取得・キャッシュ保存・描画がすべて成功した後」に限定されている (`published-data.js` の `if (result.cached)`)。

7.2 残作業 (Phase 5)

正本は minoh-hiking `docs/deployGuide-202608.md` §6.3。本節はその写しではなく、MapPublisher 側から見た進捗の把握用である。

#	作業	状態
1	<code>public/data/</code> の同梱 geojson 2件を削除	完了 (2026-08-20)
2	<code>public/closures.js</code> を削除	未 (全端末の更新完了を待つ。下記)
3	<code>vercel.json</code> の <code>/data/(.*)</code> キャッシュ設定を見直す	未
4	Vercel から <code>CLOSURES_PUBLISH_TOKEN</code> を削除	完了 (2026-08-20)
5	Blob 上の旧履歴 <code>closures/history/</code> を削除	完了 (2026-08-20)
6	MapEditor <code>docs/dataspec-geojson-202608.md</code> への注記	完了 (2026-08-20・v2.9)

1と2で影響が違う (順序を分けた理由)

どちらも「旧シェルをキャッシュしたままの端末が旧 `app.js` から参照する」点は同じだが、参照の仕方が違うため影響が大きく異なる。

対象	参照の仕方	消したときの影響
<code>data/*.geojson</code>	<code>fetch()</code>	旧 <code>map.js</code> は <code>console.warn</code> して <code>return null</code> 。表示は継続し、緊急ポイント・ルート・スポットが出ないだけ

対象	参照の仕方	消したときの影響
<code>closures.js</code>	<code>import</code> 文 (旧 <code>app.js:29</code>)	モジュール読み込みが失敗し、アプリが起動しない

このため1は先行して実施し、2は全端末がアプリ更新を通してから行う。

`tile_buffers.geojson` は対象外

`public/data/tile_buffers.geojson` (1,228,182 B) は未参照だが、**DownloadArea** が出力する成果物であり本移行とは無関係のため残す。

3. 旧履歴の削除

`closures/` をフォルダごと削除してはならない。このフォルダには **現在公開中のデータ本体**が入っている。

```

closures/
  minoh-hiking-closure.geojson  ← 現在公開中の本体。消してはいけない
  previous.geojson              ← 前回分（切り戻し用）。消してはいけない
  history/                      ← ★ 削除対象はここだけ
    2026-07-...-v2026-07-16_1.geojson
    ...

```

削除してよいのは `closures/history/` 配下だけである（パスは `api/_lib/datasets.js` の `blobPath / previousPath` を参照）。

`api/` に `closures/history/` へ書き込む処理はもう残っていない（前回分1世代 `previous.geojson` のみ）。過去に作られた blob が孤児として残っているだけである。`addRandomSuffix: true` で作成されたためパスを推測できず、URL を直接叩いて消せない。**Vercel ダッシュボード** → **Storage** → **Blob ストアのブラウザ**で `closures/history/` 配下だけを選んで削除する。

誤って本体を消した場合に起きること

`GET` は blob が無くても **200 で空の FeatureCollection を返す**（`_lib/publish.js` の `handleGet`。アプリの表示を止めないための仕様）。そのためエラーにならず、**気づきにくい形で次の状態になる**。

対象	影響
<code>manifest.json</code>	消えないため <code>version 2026-08.2 / count 7</code> のまま。実体と食い違う
既に更新済みの端末	保存済み version と manifest が一致するので 本体を取りに行かず 、キャッシュの内容を表示し続ける
未更新・新規の端末	空を取得して 通行止めが1件も出ない 。しかも version を保存するため、次の公開まで復旧しない

復旧方法: MapPublisher から通行止めデータを再公開する。新しい version (`2026-08.3`) が採番され、全端末が取得し直す。ただし再公開までの間、一部の端末で通行止め情報が欠けた状態になる。

`previous.geojson` も失われるため、切り戻し先が無くなる点にも注意する。

8. 移行手順（実施済み）

以下の順序で実施した。順序を誤ると全ユーザーの地図が消えるため、再実施・別環境への展開時と同じ順序を守ること。

1. Vercel に `MAP_PUBLISH_TOKEN` を設定する（コードのデプロイより先。未設定だと 503 / E02）
2. Phase 1 のコードをデプロイする
3. MapEditor で統合 geojson を出力する（データ修正は不要。整形は MapPublisher が行う）
4. MapPublisher で初回公開する → `mapdata 2026.1 / closures 2026-08.2`
5. `GET /api/mapdata` を直接開き、件数・type 構成・公開スキーマ・座標を確認する
6. 確認できてから Phase 4（アプリ側切替）をデプロイする
7. 1リリース分の様子を見てから `public/data/*.geojson` を削除する（ロールバック余地を残す）
8. `CLOSURES_PUBLISH_TOKEN` を Vercel から削除する
9. Blob 上の旧履歴 `closures/history/` を `del()` で一括削除する

運用者への影響: localStorage キーを `closure-editor.publish-token` から `map-publisher.publish-token` へ変更したため、切替後に公開トークンの再入力が1回だけ発生する。トークンの値そのものは変更不要（環境変数名だけが変わった）。

9. 要判断事項・既知の課題

9.0 配信データのサイズ（対応不要と判断・クローズ）

公開API は Blob へ `JSON.stringify(data, null, 2)`（インデント付き）で保存し、GET はそれをそのまま返している。生サイズは 478,894 B である。

当初これを「取得1回あたり 270KB 余計に流れている」として最適化の候補に挙げたが、**実測の結果その前提は誤りだった**。Vercel が Brotli で自動圧縮しており、実際に回線を流れるのは 45,509 B である（応答ヘッダ `Content-Encoding: br`）。

	生	gzip 圧縮後
現状（整形済み）	478,894 B	41,438 B
最小化した場合	191,759 B	36,430 B
差	-287,135 B	-5,008 B

インデントは圧縮でほぼ消えるため、削減できるのは実質 5KB 程度にとどまる。加えて本体の取得はバックグラウンドで行われ（\$5.1 の手順2で先にキャッシュを描画する）、取得が発生するのは version が変わったときだけである。

一方で最小化にはコストがある。

案	コスト
配信時に最小化	GET のたびに 479KB を parse + stringify するサーバー CPU が乗る。かえって悪化しう

案	コスト
保存時に最小化	Blob の中身をブラウザで直接開いて確認できなくなる

結論: 現状維持とする (2026-08-20 決定)。 5KB の削減のために上記のコストを払う価値は薄く、公開結果を目視確認できる利点のほうが大きい。

9.1 緊急ポイントレイヤーの filter 追加 (対応必須)

`buildEmergencyLayer` は filter を持たず、**全 feature を Point マーカーとして描画する**。統合ファイルをそのまま渡すと route も spot も緊急ポイント色で描画される。`type === 'ポイントGPS'` の filter 追加が必須。

9.2 ルート端点の未解決と spot id の不要性 (対応方針確定)

移行で触るデータ経路そのものに、既存の不具合がある。

全route 292本のうち、端点IDを解決できないもの: 54本 (18.5%)
未解決の端点名 48件 のうち 47件が spot の「name」と一致
route の端点座標フィールド: 584箇所中 583箇所が設定済み (充足率 99.8%)

原因: route は spot を **name** (滝道32鉄橋) で参照するが、`buildEndpointIndex` は **id** (spot02_滝道32鉄橋) でしか索引を作っていない。

MapEditor 側は既に正しく解決しており (`resolveCoordsById` が spot を name でも引き当て、同名が複数ある場合は相手端点に近い方を選ぶ)、結果を `startPointGPS` / `endPointGPS` に書き出している。**minoh-hiking** がそのフィールドを使っていないだけ。

結果として現在、**54本のルート線が端点スポットまで伸びず、最初／最後の間差点で切れて描画されている。**

結論: 公開データに spot の id は不要

spot は **name** と **geometry** を持ち、ルートの開始点・終了点も `startPointGPS` / `endPointGPS` に座標を持つ。スポットの表示にもルートの描画にも **spot の id は使われていない**。検証結果:

spot id を参照している route 端点: 0件 (route は spot を name で参照している)
route の端点座標フィールド: 584箇所中 583箇所が設定済み (充足率 99.8%)
minoh-hiking で spot id を使っている箇所: ポップアップの表示のみ
MapEditor の `spotEditor.js` が `properties.id` を参照している箇所: 0件

公開スキーマ (§2.2) で **spot の id を出力しない**。名称の重複 (トイレ×5、WC×4 ほか12種17件) は許容し、識別は座標で行う。

派生する効果

項目	効果
spot07_wc の ID 重複	問題そのものが消滅する。初回公開前のデータ修正が不要になる
公開API の id 一意検証	既存実装が <code>if (id != null)</code> で未設定をスキップするため、 検証コードは無変更で通る
MapPublisher の重複検出	id 前提をやめ、 読み込みは置換方式 とする (§6.5)
配信サイズ	spot の <code>id / source / description</code> 除去で -10.2%

併せて行う: 端点座標フィールドを正とする

`withRouteEndpoints` を `startPointGPS / endPointGPS` 参照に変更する（公開スキーマでは `startPoint / endPoint` を出力しないため、ID索引による解決は行わない）。

- 端点解決が 530/584 → **583/584** に改善する（残る1箇所は座標自体が存在せず、どの方式でも描けない）
- 同名 spot の曖昧性を minoh-hiking が扱う必要がなくなる。MapEditor の `resolveCoordsById` が「相手端点に近い方を選ぶ」形で**幾何的に解決済み**のため
- `buildEndpointIndex()` は不要になり削除する。`loadEmergencyPointsLayer` 内の `rebuildHikingLayer()`（読み込み順の race 対策）も併せて不要になる

併せて行う: spot ポップアップの表示

`buildHikingLayer` の spot ポップアップは **id** を太字で、その下に **name** を出している。公開データに id が無くなるため、**name のみの表示に変更する**。

MapEditor 側の出力形式は変更しない。`id / source / description` は MapEditor の作業ファイルには残し、**MapPublisher が公開時に落とす**。編集用の識別子と配信データを分離しておくほうが、双方の都合に引きずられない。

10. 影響を受けるドキュメント

リポジトリ	ドキュメント	状態 / 改訂内容
minoh-hiking	<code>docs/publish-api-202608.md</code>	作成済み 。契約2.0 と公開スキーマの 正本 （その後 2.1 で <code>tiles</code> を追加）
MapPublisher	<code>docs/funcspec-202608.md</code>	改訂済み （本移行では v1.2。現在 v1.6 ）。ClosureEditor 名の解消、編集機能の削除、2データセット公開。§6 は <code>publish-api-202608.md</code> への依存項目のみに限定
MapPublisher	<code>docs/dataspec-202608.md</code>	改訂済み （本移行では v2.2。現在 v2.5 ）。公開スキーマは正本を参照し、MapEditor から受け取る 入力ファイル の仕様と整形規則に絞った
MapPublisher	<code>docs/usersGuide-202608.md</code>	改訂済み （本移行では v1.1。現在 v1.5 ）。編集手順を削除し、ハイキングマップデータの公開手順を追加

リポジトリ	ドキュメント	状態 / 改訂内容
minoh-hiking	docs/funcspec-202607.md	§10.2（契約1.0）を publish-api-202608.md への参照に置き換える。§11 のデータファイル仕様を配信経由に改訂。 §3.4 ルート端点の補完を §9.2 の方式に改訂
MapEditor	docs/dataspec-geojson-202608.md	出力は作業用ファイルであり、公開時に MapPublisher が整形することを明記。 コードの修正は不要 （§11）

11. MapEditor は修正不要（確認済み）

公開スキーマへの整形を MapPublisher 側で行う設計にしたため、**MapEditor のコード変更は生じない**。

懸念	確認結果
出力する type	ポイントGPS / point / route / spot — MapPublisher が読む形と一致
route の端点座標	startPointGPS / endPointGPS を出力済み（584箇所中583箇所が設定済み）
spot07_wc の ID 重複	公開スキーマで spot の id を出力しないため問題にならない
reason の選択肢差（ その他 / なし）	MapPublisher は reason を素通しするだけで選択肢を持たない
closure 出力の version	MapPublisher はトップレベル version を読まない（サーバー採番）

必要なのはデータ仕様書への注記のみである。

変更履歴

版	日付	内容
1.7	2026-08-23	本書の守備範囲が 契約 2.0 まで であることを冒頭に明記し、以降の変更（契約 2.1 の tiles 追加・画面の開閉化・出力の変更）の正本を示した。§10 のドキュメント版数を現況に更新。ヘッダの版表記が 1.0 のまま変更履歴と食い違っていたのを修正
1.6	2026-08-20	配信サイズを実測（Brotli 圧縮後 45,509 B）。§9.0 を「現状維持」で決着
1.5	2026-08-20	Phase 1・3・4 の完了を反映し、稼働状況（§7.1）と残作業（§7.2）を記載。配信サイズの課題を追加（§9.0）
1.4	2026-08-18	データセットの呼称を「地図データ」から**「ハイキングマップデータ」**へ変更
1.3	2026-08-18	実装時に決める値を確定（§5.4）。SW がアプリ管理キャッシュを削除する問題を記載。MapEditor が修正不要であることの確認結果を追加（§11）
1.2	2026-08-18	公開スキーマを定義（§2.2）。 spot の id を出力しない方針に変更し、 spot07_wc の重複対応を不要化。配信サイズ -34.8%

版	日付	内容
1.1	2026-08-18	件数下限チェックを不採用に決定 (§5 の方針を維持)。履歴を前回分1世代に決定。ルート端点バグの対応方針を確定
1.0	2026-08-18	初版